

JAVA 17 OCP · YDS STUDY SERIES

Unit 03 Grammar Notes · Making Decisions

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu notlar Unit 03 review metnindeki teknik soru kalıpları ve control-flow açıklamalarında geçen grammar yapılarını ele alır.

1. `how + clause`

Kısa açıklama

`how` burada yöntem değil sıra/biçim bildirir ve noun clause başlatır.

EN: Print the elements in reverse order from how they are declared. **TR:** Elementleri bildirildikleri sıranın tersine yazdır.

YDS tip: `how they are declared` passive bir noun clause'dur; “nasıl bildirildiklerinden” yerine bağlama uygun olarak “bildirildikleri sıra” doğal olabilir.

2. `provided + clause`

Kısa açıklama

`provided`, “şu koşulla ki / -mesi şartıyla” anlamında condition bağlacı olabilir.

EN: `var` is supported, provided the compiler determines a compatible type. **TR:** Compiler uyumlu bir type belirlediği sürece `var` desteklenir.

Formül: `main clause + provided (that) + condition`

3. `once + clause`

Kısa açıklama

Bir olay gerçekleştiğinde başlayan sonucu anlatır; teknik metinde “-dığına” diye çevrilir.

EN: An object is unreachable once no live reference points to it. **TR:** Hiçbir live reference onu göstermediğinde object unreachable olur.

4. `if` ile gerçek koşul, `whether` ile dolaylı soru

`if`, code'daki condition'ı; `whether`, iki olasılık arasındaki dolaylı soruyu anlatır.

EN: Check whether the loop terminates if the value is unchanged. **TR:** Değer değişmezse loop'un sonlanıp sonlanmadığını kontrol et.

Common mistake: `whether` burada “eğer” değil, “-ip -mediği” anlamındadır.

5. `so + clause` ile sonuç

Kısa açıklama

Bir önceki durumun sonucunu “bu nedenle/böylece” ilişkisiyle verir.

EN: The variable is out of scope, so the code does not compile. **TR:** Variable scope dışındadır; bu nedenle kod derlenmez.

6. `would have + V3` · unreal past result

Kısa açıklama

Geçmişte gerçekleşmeyen bir condition'ın varsayımsal sonucunu anlatır.

EN: If the syntax had been corrected, the code would have printed 11. **TR:** Syntax düzeltilmiş olsaydı kod 11 yazdırmış olurdu.

Formül: `If + past perfect, would have + V3`

YDS tip: Gerçekte syntax düzeltilmemiş ve output oluşmamıştır. Bu yapı gerçek sonucu değil, counterfactual sonucu verir.

7. `since` ile neden

EN: The body is entered at least once since the condition is checked afterward. **TR:** Koşul sonradan kontrol edildiği için gövdeye en az bir kez girilir.

Bu cümle girilen bir `do/while` içindir; `at least once` alt sınır verir, “tam bir kez” anlamına gelmez. Gövde içindeki `break`, `return` veya exception koşul değerlendirmesine ulaşılmasını engelleyebilir.

`since` başlangıç zamanı da anlatabilir; ardından bir neden clause'u geldiğinde “-dığı için” çevirisini kontrol et.

8. `unless` ve `as long as` ile koşul

Kısa açıklama

`unless`, “-medikçe / ... olmadığı sürece” anlamıyla olumsuz condition kurar. `as long as` ise bir durum devam ettiği sürece main clause'un geçerli olduğunu gösterir.

Formüller:

- `main clause + unless + affirmative clause`
- `main clause + as long as + clause`

EN: A Java 17 switch expression needs `default` unless every enum constant is covered. **TR:** Java 17 switch expression, bütün enum sabitleri kapsamıyorsa `default` gerektirir.

Bu kural standart Java 17 switch expression içindir; statement aynı zorunluluğu taşımaz. `String` veya integral selector için `default` gerekir.

EN: The loop continues as long as the expression evaluates to true. **TR:** Expression `true` ürettiği sürece loop devam eder.

YDS tip: `unless` içeren clause'u ayrıca `not` ile olumsuz yapmak çoğu bağlamda çift olumsuzluğa yol açar. Önce “if ... not” dönüşümünü kontrol et.

Cümleyi parçalayarak okuma

[İlgili kaynak bölümü](#). Aşağıdaki çalışma cümlesi kaynak bağlamına dayanır; gerektiğinde öğretim amacıyla sadeleştirilmiştir.

English: The body is entered at least once since the condition is checked afterward.

Çözümleme: *The body* = özne; *is entered* = edilgen ana yüklem. *since* neden yan cümlesini başlatır; onun öznesi *the condition*, yüklemi *is checked* tir. *at least once* sayı alt sınırı; *afterward* olay sırasıdır.

Doğal Türkçe: Koşul sonradan kontrol edildiği için gövdeye en az bir kez girilir.

Kapalı kitap kontrolü: *since* i *although* yapınca neden ilişkisi mi karşıtlık mı oluşur? *once* ve *at least once* arasında hangi teknik fark var?

Mini quiz

- 1 *provided* kullanarak *var* için bir enhanced-for kuralı yaz.
- 2 *whether* ile infinite loop hakkında dolaylı soru kur.
- 3 *If the condition had changed, ...* cümlesini tamamla.
- 4 *so* kullanarak scope ile compilation sonucunu bağla.
- 5 *unless* kullanarak *default* branch kuralını yaz.

Cevaplar

- 1 *var* is valid, provided the element type can be inferred.
- 2 Determine whether the loop terminates.
- 3 Örnek: *If the condition had changed, the loop would have terminated.*
- 4 The variable is out of scope, so the code does not compile.
- 5 A Java 17 switch expression needs *default* unless every enum constant is covered.

Kısa tekrar özeti

- Biçim/sıra noun clause'u: *how* + *clause*
- Koşul: *provided* (*that*) + *clause*
- Gerçekleşme zamanı: *once* + *clause*
- Dolaylı ikili soru: *whether* + *clause*
- Sonuç: *so* + *clause*
- Gerçekleşmemiş geçmiş sonuç: *would have* + V3
- Olumsuz koşul: *unless* + *affirmative clause*
- Sürdüğü müddetçe geçerli koşul: *as long as* + *clause*